INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KOTTAYAM

(An Institute of National Importance)

IOE-414: DEEP LEARNING

MINOR PROJECT REPORT

Project Title:
PatchTST Replication for Long Term Forecasting

Group Members:
1. Vibhaas Nirantar Srivastava (2023BCS0037)
2. Jayanth P (2023BCS0040)
3. Vinayak Sharma (2023BCS0002)
4. Vishwanath Darur (2023BCS0057)

Department of Computer Science and Engineering

October–2026

Table of Contents

Abstract 3

Base Paper Details 3

Problem Statement 3

Method Overview with Architecture 4

Dataset Details 6

Results and Visualizations 6

Implementation and Dataset Links 9

References 10

Abstract

We checked the PatchTST paper results using the authors' code on one RTX 5050 laptop GPU. The model reads a time series as small patches, and the same model is used for each variable. We ran the ETT datasets, Weather, Electricity, Traffic, ablation tests, and a short masked pre-training test. The ETT PatchTST scores were within 3% MSE of the paper. On ETTh1 with a 96-step forecast, we got 0.376 MSE while the paper has 0.375. Removing patching and channel independence made the result worse. For Traffic, our 64-channel training setup got 0.392 MSE when tested on all 862 channels. The results are useful even though our training was shorter than the paper's setup.

Base Paper Details

- **Base Paper Title:** A Time Series is Worth 64 Words: Long-term Forecasting with Transformers
- **DOI link of the paper:** https://doi.org/10.48550/arXiv.2211.14730
- **Conference/Journal Name:** International Conference on Learning Representations (ICLR)
- **Authors:** Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, Jayant Kalagnanam
- **Year of Publication:** 2023

The main idea in the paper

The title compares a long time series to a sentence. In a sentence, a word gives a small unit of meaning. A single temperature or electricity reading does not tell us much by itself. A short stretch of nearby readings can show a rise, a fall, or a repeated shape. Nie and the other authors use such stretches as the tokens given to a Transformer. The number 64 in the title is an illustration of this idea; the exact patch count depends on the input length and the patch settings.

The paper also questions whether a complicated Transformer is really needed for forecasting. A simple linear method had already given strong results on common datasets. PatchTST keeps the useful part of attention, which is comparing different parts of a sequence, but changes what the attention sees. The two choices are patching and channel independence. The authors also try learning from masked patches before training a forecast head. In our project we tested the main supervised results and a smaller version of that pre-training idea.

Problem Statement

Long-term forecasting means using past values to predict many future values. A normal Transformer can treat each time step as one token, but this becomes costly when we give it a long history. There are also many variables in datasets like weather and electricity. The paper asks if a Transformer can work better by reading short groups of values and by

sharing one model across the variables. Our project checks these main claims with a small GPU and limited training time.

What we wanted to check

A forecast has two lengths. The look-back is the past window given to the model, and the horizon is the number of future points it must predict. For example, with look-back 336 and horizon 96, we show 336 past readings and compare the output with the next 96 real readings. A 720-step horizon is harder because the answer is much farther away. We wanted to see whether the same PatchTST setup could stay close to the paper over these different horizons, not only one easy case.

Attention in a normal Transformer compares every input token with every other token. If each time point is a token, a long history makes the attention map large. Its size grows roughly with the square of the token count. Multivariate data adds another question: should the model mix temperature, load, or other variables inside attention, or should it learn a shared rule for each series? The paper argues for fewer patch tokens and for sharing one model across channels. We tried to check both decisions separately in the ablation runs.

Our goal was a useful replication on a laptop GPU. We could not train every model and dataset for as long as the paper did. So we kept the authors' data splits and test sets, made a shorter run plan, and recorded where the setup differed. We treat a close MSE and the same trend in the ablation as evidence for the idea, but we do not call it an exact reproduction of all of the paper.

Method Overview with Architecture

PatchTST cuts each variable into patches of 16 values with stride 8. Each patch becomes one input token. A shared Transformer then learns from these tokens, and a prediction head gives the future values. “Channel independence” means the same weights are used for every variable, though each variable is processed separately. This keeps the attention smaller and lets the model look at more past data.

Why patches help

We used patches of 16 values and moved forward by 8 values each time. This means two neighbouring patches overlap by 8 points. The overlap keeps the edge of a pattern from being lost when it falls between two patches. With a 336-step input, the end padding in the code gives 42 patches. Attention therefore compares 42 patch tokens for each channel rather than 336 separate points. A 42 by 42 attention map has about 64 times fewer entries than a 336 by 336 one. This is only a comparison of attention map size, not a claim that the whole training run is 64 times faster.

Each patch is first turned into a learned feature vector by a linear layer. Position information is added so that the encoder can tell earlier and later patches apart. Then a stack of Transformer encoder layers uses self-attention to compare the patches.

What channel independence means here

The input tensor has a batch, a time dimension and a channel dimension. In the official model code, the batch and channel dimensions are folded together before the encoder. So the same encoder works on each channel as if it were one separate series. The weights are shared, but the values from two different channels do not attend to each other. After attention, a shared forecast head maps the patch features to the required horizon. This is why the model can be trained with some Electricity channels and still be run on all channels at test time.

There is a trade-off in this choice. Sharing the weights gives the model many training examples and avoids having one large attention problem across all variables. At the same time, it cannot directly learn that one channel moves together with another channel inside its attention layers. Our tests support channel independence on ETTh1, but they do not prove it is always best for every type of multivariate data.

Past values -> 16-step patches -> shared Transformer -> future values

We used a 336-step look-back for the main runs and forecast 96, 192, 336 or 720 steps. We also tested a masked patch pre-training run, where 40% of the patches are hidden and the model learns to fill them. Our code comes from the paper authors, with small fixes for current libraries. Training epochs and some training windows were reduced to fit the laptop.

Normalization, baseline and masked learning

The model also uses RevIN. It normalizes each sample and channel using the statistics of the input window, then reverses this change on the forecast. This is useful when a series has a different level or scale from another one. In our settings the RevIN affine option was off. For the main ETT runs we used a smaller three-layer model on the hourly sets and a wider three-layer model on the minute sets. Weather, Electricity and Traffic also used the wider model.

We used DLinear as a simple comparison. It first takes a moving average to get a smoother trend, leaving the rest as a residual part. It then uses linear layers to extend both parts into the future and adds the two outputs. It has no attention. This makes it a good baseline: if PatchTST only matches DLinear, the extra model complexity would need a good reason.

In the self-supervised part, the model gets 512 past values, made into non-overlapping patches of 12. We hide 40% of the patches and train it to reconstruct the missing values. After that, a linear probe trains only a small forecast head while the encoder is frozen. Fine-tuning later lets the encoder change too.

Dataset Details

ETTh1, ETTh2, ETTm1 and ETTm2 are transformer temperature datasets with seven variables each. ETTh is hourly and ETTm is recorded every 15 minutes. Weather has 21 variables, Electricity has 321 and Traffic has 862. We used the standard train, validation and test split from the authors' code. For PatchTST on ETTm, Weather and Traffic we used every second training window, but kept the full validation and test sets. In Electricity and Traffic we also checked how a shared model trained on fewer channels works on all channels at test time.

How the training examples were made

The ETT datasets gave us a smaller starting point: seven variables, recorded either hourly or every 15 minutes. Weather has 21 variables, Electricity has 321 and Traffic has 862. We used the CSV files expected by the authors' loader and its chronological train, validation and test split. The scaler is fitted on the training part and the same transform is applied to later parts. This is important because using information from the test period while scaling would make the result too optimistic.

The loader moves a window through each split to make many examples. The past part becomes the input and the next 96, 192, 336 or 720 points become the target. We used 336 past points for most runs, with separate look-back experiments at 96, 192, 512 and 720. PatchTST/64 used a 512-step look-back for its ETT settings. For longer ETTm, Weather and Traffic training, we used every second training window so one epoch was shorter. The validation and test splits stayed the same.

For Electricity we picked 32 of the 321 channels with a fixed random seed during one training setup, then tested the shared weights on all 321. We also trained on every Electricity channel. For Traffic we trained on 64 of 862 channels and tested on all 862. These subset tests show that the weights can still be used on unseen channels. The training schedules also differ, so they are not a controlled measure of channel count alone.

Results and Visualizations

Table 1 shows the 96-step test MSE. Smaller MSE means the forecast is closer to the real data. PatchTST/42 beat our DLinear run on all four ETT sets and on Weather. The Paper column gives the paper's PatchTST/42 result. The Electricity 32-channel row is our own setup, so there is no paper number for that row. For Traffic, our model trained on 64 channels, while the paper number uses the full training setup. We did not run DLinear on Traffic.

We measure errors with MSE and MAE. MSE squares each error and is more sensitive to large misses, while MAE uses absolute differences. Both are better when lower. We compare values only for the same dataset and horizon, because scales differ.

Table 1. Test MSE for a 96-step forecast

Dataset	PatchTST ours	Paper	DLinear ours
ETTh1	0.376	0.375	0.438
ETTh2	0.276	0.274	0.300
ETTm1	0.292	0.290	0.300
ETTm2	0.163	0.165	0.173
Weather	0.155	0.152	0.175
Electricity 32 ch	0.145	—	0.142
Electricity full	0.137	0.130	0.142
Traffic 64 ch	0.392	0.367	—

Comparison across forecast lengths

The first table shows one horizon, but we ran PatchTST/42 at four horizons on each ETT dataset. All 16 ETT MSE values were within about 3% of the paper's PatchTST/42 values. On ETTh1, for example, our MSE moved from 0.376 at horizon 96 to 0.455 at horizon 720; the paper gives 0.375 and 0.449. On ETTm2 the same two horizons were 0.163 and 0.366 for us, compared with 0.165 and 0.367 in the paper. This closeness is the strongest replication result we got.

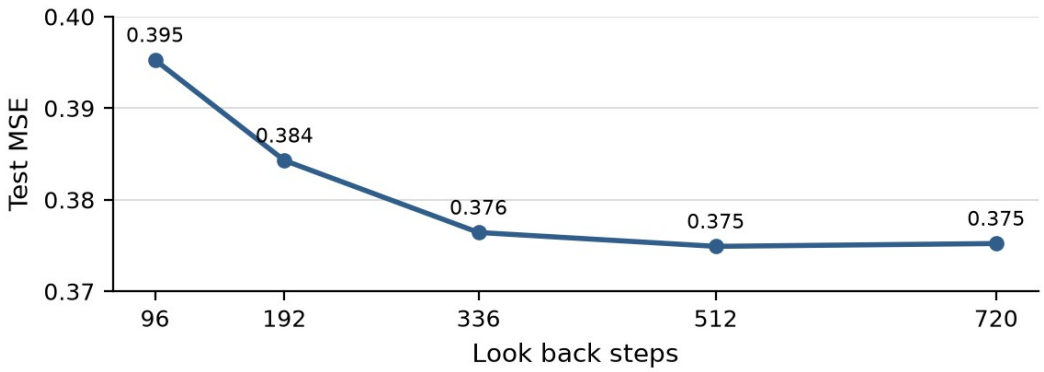
Our DLinear run was worse than PatchTST on the four ETT datasets at horizon 96. The gap on ETTh1 was clear, 0.438 against 0.376 MSE. The gap on ETTm1 was smaller, 0.300 against 0.292. DLinear did not match some of the paper's hourly results as closely under our shorter schedule. We think training time and settings may matter there, but we did not isolate one cause. The comparison is still useful because both models were run on our machine and evaluated with the same test split.

In ETTh1 at horizon 96, the full model got 0.376 MSE. Without patching it got 0.387, without channel independence it got 0.416, and without both it got 0.484. This supports the two design choices. The look-back graph below shows that a longer history helps, mainly until about 336 steps.

For the ablation, we made four versions. The full model has patches and channel independence. CI only keeps the separate-channel design but makes every single point its own token. P only keeps patches but places all channels together inside a token, using the channel-mixing model that we added. The last version has neither choice. At horizon 336 the MSE values followed the same order: 0.432 for full, 0.452 for CI only, 0.499 for P only and 0.526 for neither. This makes the result less likely to be just one lucky 96-step forecast.

Our channel-mixing ablation follows the idea described in the paper, but the code for it was written for this project because the official supervised repository did not include that variant. That matters when comparing exact digits with the paper's ablation table. We read the ordering of the models as the clearer result here: taking away either patching or separate-channel processing made the ETTh1 error go up.

Figure 1. ETTh1 96-step forecast MSE for different look-back lengths



The shared model could be tested on all 321 Electricity channels after seeing 32 in training, and on all 862 Traffic channels after seeing 64. We also tried full-channel Electricity training and a short masked pre-training test. Their results are discussed below.

What the look-back and larger datasets show

The look-back graph uses the same ETTh1 96-step target while changing only how much past data is supplied. MSE went from 0.395 with 96 past points to 0.384 at 192 and 0.376 at 336. It was about 0.375 at both 512 and 720. A longer history helped at first, which fits the reason for using patches, but the last two increases in context gave almost no further improvement in our run. More context is not automatically useful forever.

Weather was a larger supervised check. Our PatchTST MSE was 0.155 at horizon 96 and 0.252 at 336; the paper reports 0.152 and 0.249. At horizons 192 and 720 we got 0.201 and 0.320. Electricity was harder on the small GPU. The 32-channel training setup got 0.145 MSE and the full-channel one got 0.137, against the paper's 0.130. The full model needed a 60-minute cap. Traffic had 862 channels, so we trained on 64 of them and tested on all 862. At horizon 96 we got 0.392 MSE and 0.271 MAE; the paper reports 0.367 and 0.251. The gap is about 6.9% on MSE. We used fewer training channels and every second training window, so this is not the same training setup as the paper.

Pre-training result and limits

The short masked pre-training test produced a 0.368 MSE linear probe on ETTh1 at horizon 96; the paper reports 0.371. Fine-tuning gave 0.370 for us and 0.366 in the paper. At horizon 336 the probe was 0.445, equal to the paper's rounded MSE, while our fine-tuned run was 0.428 against 0.450. The numbers look encouraging, but these are only two forecast lengths after 20 pre-training epochs. We did not test transfer to another dataset, which is a larger claim in the paper.

We used one random seed per setting and shortened several schedules to fit the laptop time budget. We therefore do not have error bars from repeated runs. Full Electricity stopped at its time cap. Traffic completed 30 epochs, but its smaller training setup limits a direct number-to-number comparison with the paper. The main patterns are still consistent: ETT supervised scores were close, both design choices helped in the ETTh1 ablation, and the look-back improvement levelled off after a while.

Implementation and Dataset Links [GitHub]

Code path and changes

We started from the authors' official supervised and self-supervised code instead of writing PatchTST again. This kept the main encoder and forecasting head close to what the paper used. We made small compatibility changes for the current PyTorch, NumPy and pandas versions, and removed fixed local data paths. The list of changes to upstream files is in UPSTREAM_PATCHES.md. We also added options for a time cap, a channel subset and a stride through training windows. These were for running the project on the available machine; they were not new claims about the model.

The separate-channel model already came from the official code. To test the P-only and neither cases, we added PatchTST_CM. It makes one token from the same time patch of all channels, instead of treating each channel as its own series. With patch length 1 it becomes the point-token, channel-mixing version. We kept this extra implementation separate from the authors' PatchTST model so the main replication still uses their original path.

Training and record keeping

Our runner starts the planned jobs one at a time and gives each one a name such as A/PatchTST42/ETTh1/96. It sends the job to the correct upstream training script, saves a log, and writes the final MSE and MAE to a CSV file. If a job already has a completed row, the plan skips it on restart. This was helpful because we could stop after a time limit and later continue without repeating the completed runs.

The supervised code trains with Adam and MSE loss. A learning-rate schedule and validation loss guide training, and early stopping keeps the best checkpoint. We allowed at most 30 epochs for PatchTST rather than the longer paper schedule, and a per-job cap could stop some runs sooner. We used mixed precision on Weather, Electricity and Traffic to fit the GPU memory and time. Traffic training used batch size 24, but testing all 862 channels at that size did not fit GPU memory. We loaded the best checkpoint and tested with batch size 1, adding the errors batch by batch. This changed only how we measured the saved model. The runner kept each result in a CSV file beside the comparison with the paper.

Our implementation and result CSVs: https://github.com/vibhaas/patchtst-replication. Authors' code: https://github.com/yuqinie98/PatchTST. Dataset mirror: https://huggingface.co/datasets/thuml/Time-Series-Library. The comparison file in our repository lists every completed result beside the paper number. The biggest limit is shorter training than the paper, especially on the larger datasets.

What we take from the project

For us, the most convincing part was not one single lowest number. It was that the ETT results stayed close to the paper across many horizons, while the ablation moved in the expected direction. Patches reduce the attention length, and channel independence lets one set of weights work across variables. We also saw that the extra history helped most up to around 336 points on ETTh1. Electricity and Traffic showed that the same weights can be applied to channels left out of training, though the Traffic score was farther from the paper. A next step would be to rerun the larger datasets with the paper's full schedule and more than one seed.

References [APA Style]

Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). A time series is worth 64 words: Long-term forecasting with transformers. International Conference on Learning Representations. https://doi.org/10.48550/arXiv.2211.14730

Nie, Y. (2023). PatchTST [Source code]. GitHub. https://github.com/yuqinie98/PatchTST

THUML. (n.d.). Time-Series-Library [Data set]. Hugging Face. https://huggingface.co/datasets/thuml/Time-Series-Library